

Reconstruir o InvestiGator de raiz

Dez fases, com o código que está mesmo lá

Henrique José da Silva Santos

MEIA, Instituto Superior de Engenharia do Porto

Guia de estudo

O outro guia ensina-te a **defender**: os números de cor, as perguntas prováveis, o que não dizer. Este ensina-te a **construir**.

A diferença importa numa prova.

Uma pergunta sobre *quanto deu* responde-se com um número.

Uma pergunta sobre *como fizeste* não.

São dez fases, pela ordem em que o sistema foi realmente construído. Cada uma traz:

- o **problema** que ela resolve, numa frase;
- o **código real**, copiado dos ficheiros e não reescrito;
- **a linha que interessa**, ou seja onde é que a garantia é mesmo feita;
- **como se verifica**, com um comando que podes correr.

Os erros que cada fase custou estão reunidos no penúltimo *slide*, de propósito: lidos de seguida, são a parte deste trabalho que mais se aprende.

Fase 1: de preços para retornos

\$500 é muito numa empresa e pouco noutra: o sistema trabalha em **retornos**, e tenta cinco fontes gratuitas por ordem fixa.

investigator/market_data/prices.py

```
def get_price_history(ticker: str, period: str = "6mo", interval: str = "1d") -> pd.DataFrame:
    # ...
    try:
        df = _yf_history(ticker, period=period, interval=interval)
        _LAST_SOURCE[ticker.upper()] = "yfinance"
        return df
    except Exception as exc: # noqa: BLE001
        if interval != "1d":
            raise RuntimeError(f"Sem dados de preços para o ticker '{ticker}'") from exc
    # ...
    df, fonte = fallback_daily(ticker, start, end)
    _LAST_SOURCE[ticker.upper()] = fonte
    print(f"[precos {ticker}] yfinance indisponível → servido por {fonte} ({len(df)} dias)")
    return df
# ...
def log_returns(close: pd.Series) -> pd.Series:
    """Retornos logarítmicos a partir da série de preços de fecho."""
    return np.log(close / close.shift(1)).dropna()
```

A linha que interessa. O `shift(1)`. Cada retorno divide o fecho de hoje pelo do dia **anterior** e nunca por um posterior: nenhum valor futuro entra na norma que a fase seguinte constrói.

Verifica-se com. `pytest tests/test_prices_fallback.py`

Fase 2: o que conta como movimento invulgar

2% é banal na Tesla e enorme na Johnson & Johnson, portanto um limiar fixo em percentagem mediria *volatilidade* e não *raridade*. Mede-se em desvios-padrão face aos últimos vinte dias da própria empresa.

```
investigator/anomaly_detector/detector.py
```

```
def _score(last: float, mu: float, sigma: float, threshold: float) -> tuple[float, bool, bool]:
    # ...
    if sigma > 0.0:
        z = (last - mu) / sigma
        return z, abs(z) > threshold, False
    moved = not math.isclose(last, mu, rel_tol=0.0, abs_tol=1e-15)
    return 0.0, moved, True
    # ...
    recent = r.iloc[-window - 1 : -1] # janela ANTES do último (sem lookahead)
    last = float(r.iloc[-1])
    mu = float(recent.mean())
    sigma = float(recent.std(ddof=1))
    z, is_anomaly, zero_variance = _score(last, mu, sigma, threshold)
    # ...
    zero_variance=zero_variance,
```

A linha que interessa. O `-1` do fim da fatia `r.iloc[-window - 1 : -1]`, que **exclui o próprio dia julgado**. Com `r.iloc[-window:]` o salto entrava no cálculo da sua própria régua, inflacionava o sigma e encolhia o seu próprio z. Se a norma for constante, repetir o valor não dispara; afastar-se dele dispara, mas o produto diz que o z é indefinido.

Verifica-se com. `pytest tests/test_anomaly_detector.py`

Fase 3: deitar fora o que não é da empresa

A fonte etiqueta mal. Chegavam notícias de escritórios de advogados etiquetadas como *AMD*, e listas de *top movers* repetidas para vários tickers. Sem este filtro, 67% do que entra é ruído, e um precedente construído sobre ruído é pior do que nenhum.

```
investigator/news_fetcher/relevance.py
```

```
def is_relevant(headline: str, ticker: str) -> bool:
    """True se a manchete é plausivelmente SOBRE esta empresa (e não boilerplate de mercado)."""
    if not headline or not headline.strip():
        return False
    low = headline.lower()
    if _BOILERPLATE_RE.search(low):
        return False
    terms = [ticker.lower()] + COMPANY_NAMES.get(ticker.upper(), [])
    return any(_mentions(low, t) for t in terms)
```

A linha que interessa. A ordem. O `_BOILERPLATE_RE` corre **antes** da procura de menções, e tem de correr: uma lista de *top movers* menciona a empresa, portanto passaria a última linha sem dificuldade. Trocar as duas deixaria entrar exactamente o ruído que este filtro existe para apanhar, e nada avisaria. Repare-se ainda no `COMPANY_NAMES`: procurar só o *ticker* perderia toda a notícia que escreve *Nvidia* e nunca *NVDA*.

Verifica-se com. `pytest tests/test_relevance.py`

Fase 4: uma frase como um ponto no espaço

Duas notícias sobre o mesmo assunto raramente partilham palavras. Um codificador de frases já treinado transforma cada título em 384 números, de forma a que proximidade entre pontos seja proximidade de *significado*.

```
investigator/historical_kb/onnx_embedder.py
```

```
def masked_mean_pool(hidden: np.ndarray, attention_mask: np.ndarray) -> np.ndarray:
    """Mean pooling com máscara + normalização L2 --igual ao sentence-transformers.

    hidden: (n, seq, dim); attention_mask: (n, seq) com 1 nos tokens reais. Puro numpy,
    testável offline (tests/test_onnx_embedder.py cobre a matemática sem o modelo).
    """
    mask = attention_mask[..., None].astype("float64")
    summed = (hidden.astype("float64") * mask).sum(axis=1)
    counts = np.clip(mask.sum(axis=1), 1e-9, None)
    pooled = summed / counts
    norms = np.linalg.norm(pooled, axis=1, keepdims=True)
    return pooled / np.clip(norms, 1e-12, None)
```

A linha que interessa. A última linha, a normalização. Depois dela todos os vectores têm comprimento 1, e por isso o cosseno entre dois passa a ser um simples produto interno. Não é uma optimização: é o que faz com que *semelhança* deixe de depender do comprimento do título.

Verifica-se com. `pytest tests/test_onnx_embedder.py`

Fase 5: encontrar o precedente

Com cada título já como um ponto, *parecido* passa a ser uma conta: o cosseno do ângulo entre dois vectores. Um resultado perto de 1 quer dizer que apontam para o mesmo sítio.

```
investigator/correlation_engine/similarity.py
```

```
def cosine_similarity(a, b) -> float:
    """Similaridade do cosseno entre dois vetores 1D. Vetor nulo → 0.0 (sem direcção)."""
    a = np.asarray(a, dtype="float64")
    b = np.asarray(b, dtype="float64")
    na = float(np.linalg.norm(a))
    nb = float(np.linalg.norm(b))
    if na == 0.0 or nb == 0.0:
        return 0.0
    return float(np.dot(a, b) / (na * nb))
```

A linha que interessa. A guarda `if na == 0.0 or nb == 0.0`. Um vector nulo não tem direcção, logo o ângulo não existe e a divisão rebentaria. Devolver 0.0 diz *não se parece com nada*, que é a leitura correcta, e mantém a função total: nenhuma consulta pode derrubar o sistema por causa de um registo degenerado.

Verifica-se com. `pytest tests/test_similarity.py`

Fase 6: medir o que aconteceu depois

Esta é a fase que separa este trabalho de uma previsão. Mede-se o que o preço **fez** nos 1, 3 e 5 dias a seguir a uma notícia *passada*. É observação verificável, não projecção.

```
investigator/correlation_engine/event_study.py
```

```
def post_event_returns(close: pd.Series, event_idx: int,
                      horizons: tuple[int, ...] = (1, 3, 5)) -> dict[int, float]:
    # ...
    p0 = float(close.iloc[event_idx])
    out: dict[int, float] = {}
    for h in horizons:
        j = event_idx + h
        out[h] = float(close.iloc[j] / p0 - 1.0) if 0 <= j < len(close) else float("nan")
    return out
# ...
def abnormal_returns(close: pd.Series, market_close: pd.Series, event_idx: int,
                    horizons: tuple[int, ...] = (1, 3, 5)) -> dict[int, float]:
    # ...
    own = post_event_returns(close, event_idx, horizons)
    mkt = post_event_returns(market_close, event_idx, horizons)
    return {h: own[h] - mkt[h] for h in horizons}
```

A linha que interessa. O `float("nan")` quando `j` sai da série. Uma notícia dos últimos cinco dias **ainda não tem** desfecho a `+5`; devolver zero, ou o último preço disponível, inventaria um resultado. O NaN propaga-se e obriga quem consome a decidir o que fazer com a ausência.

Verifica-se com. `pytest tests/test_event_study.py`

Fase 7: foi a empresa, ou foi o mercado?

Um movimento reparte-se em três: o que veio do mercado inteiro, o que veio do setor, e o que sobra e é da própria empresa. Os betas são estimados só com dias **anteriores**.

```
investigator/correlation_engine/decomposition.py
```

```
def _shrink(beta_raw: float, std_err: float, prior: float,
            prior_sd: float = PRIOR_BETA_SD) -> float:
    """Encolhe 'beta_raw' na direção de 'prior', pesando pela precisão da estimativa."""
    if not np.isfinite(std_err) or std_err <= 0:
        return beta_raw # ajuste exato: nada a encolher
    w = prior_sd**2 / (prior_sd**2 + std_err**2)
    return w * beta_raw + (1.0 - w) * prior

# ...
total = float(t[-1])
r_m_today = float(m[-1])
r_s_today = float(s[-1]) if s is not None else 0.0

# Janela de estimação: estritamente ANTES do dia explicado (anti-lookahead).
hist_t, hist_m = t[:-1][-window:], m[:-1][-window:]
hist_s = s[:-1][-window:] if s is not None else None
```

A linha que interessa. O peso w do `_shrink`. Um beta estimado em vinte dias é ruidoso, e numa janela agitada a AMD deu $\beta = 4.43$. Cortar em ± 4 seria outra constante por justificar; aqui o encolhimento é **proporcional à precisão da estimativa**: com erro-padrão pequeno o beta fica intacto, com erro grande aproxima-se da referência.

Verifica-se com. `pytest tests/test_decomposition.py`

Fase 8: um conjunto de treino sem o futuro lá dentro

É aqui que os trabalhos com dados temporais se enganam, e o engano não dá erro: dá um resultado bom de mais.

```
investigator/triage/dataset.py
```

```
def assign_splits(dates: pd.Series, train_frac: float = 0.70, val_frac: float = 0.15,
                  embargo_days: int = 5) -> pd.Series:
    """Divisão TEMPORAL por dias únicos (70/15/15) com embargo entre blocos.

    Todas as linhas do mesmo dia ficam no mesmo bloco (evita fuga por notícias do mesmo dia).
    O embargo remove 'embargo_days' dias únicos APÓS cada fronteira (rotulados "embargo",
    excluídos do treino e da avaliação) --protege contra sobreposição das janelas de rótulo
    ( $h \leq 5$ ) entre blocos.
    """
    d = pd.to_datetime(pd.Series(dates).reset_index(drop=True))
    unique_days = np.array(sorted(d.unique()))
    n = len(unique_days)
    i1 = int(n * train_frac)
    i2 = int(n * (train_frac + val_frac))
    train_days = set(unique_days[:i1])
    emb1 = set(unique_days[i1 : i1 + embargo_days])
    val_days = set(unique_days[i1 + embargo_days : i2])
    emb2 = set(unique_days[i2 : i2 + embargo_days])
```

A linha que interessa. `unique_days`, e não as linhas. Dividir por linhas poria duas notícias **do mesmo dia**, com o mesmo rótulo, uma no treino e outra no teste: o modelo veria a resposta. O embargo trata do segundo problema, o rótulo olhar cinco dias em frente.

Verifica-se com. `pytest tests/test_triage_dataset.py`

Fase 9: treinar, e depois calibrar

Um modelo dá uma pontuação, e uma pontuação não é uma probabilidade. Para o alerta poder dizer 39% é preciso um segundo ajuste, feito num bloco reservado que o modelo nunca viu.

```
investigator/triage/model.py
```

```
class PlattCalibrator:
    """p_calibrada = sigmóide(a·score + b), com (a,b) ajustados na validação."""

    a: float
    b: float

    def __call__(self, scores: np.ndarray) -> np.ndarray:
        z = self.a * np.asarray(scores, dtype="float64") + self.b
        return 1.0 / (1.0 + np.exp(-z))

def fit_platt(scores_val: np.ndarray, y_val: np.ndarray, seed: int = 42) -> PlattCalibrator:
    lr = LogisticRegression(max_iter=1000, random_state=seed)
    lr.fit(np.asarray(scores_val, dtype="float64").reshape(-1, 1), y_val)
    return PlattCalibrator(a=float(lr.coef_[0][0]), b=float(lr.intercept_[0]))
```

A linha que interessa. O `scores_val`, no nome do argumento. A calibração ajusta-se na **validação** e nunca no treino: ajustada no treino, corrigiria o sobreajustamento em vez da escala, e as probabilidades sairiam confiantes de mais onde o modelo está errado. Aqui $a = 3.700$ e $b = -2.313$.

Verifica-se com. `pytest tests/test_triage_model.py`

Fase 10: observar o que corre, e deixar-se contrariar

Sem esta fase o trabalho acabava numa métrica de teste. Cada decisão real é registada, e dias depois confrontada com o que o preço *fez*. Foi assim que se soube que a porta **não estava a ajudar**.

investigator/triage/postval.py

```
rep: dict = {
    "n": n,
    "n_mantidas": len(kept),
    "precisao_mantidas": (float(np.mean([d["label"] for d in kept])) if kept else float("nan")),
    "n_suprimidas": len(dropped),
    "precisao_suprimidas": (float(np.mean([d["label"] for d in dropped]))
                           if dropped else float("nan")),
    "base_rate": (float(np.mean([d["label"] for d in labeled])) if labeled else float("nan")),
    "brier": (float(np.mean([(d["prob"] - d["label"]) ** 2 for d in with_prob]))
             if with_prob else float("nan")),
    "calibracao": [],
}
```

A linha que interessa. A *precisao_suprimidas*. Comparar as mantidas com a taxa-base responde a *a porta acerta?*, que é a pergunta fácil. A pergunta que interessa é *a porta escolhe melhor do que aquilo que deitou fora?*, e essa exige medir também as suprimidas. Medido: mantidas 0.589 contra suprimidas 0.617, ou seja o que a porta rejeitou era *mais* material.

Verifica-se com. `python scripts/post_validate.py`

Lidos de seguida, são a parte deste trabalho que mais se aprende, e são a melhor resposta a “*o que é que correu mal?*”. Nenhum deles deu erro nenhum.

fase	o erro, e porque é que passou despercebido
1. preços	A última fonte da cadeia devolvia um resultado <i>vazio</i> como sucesso. Preços vazios propagavam-se em silêncio, sem uma única excepção. Numa cadeia de recurso, a resposta vazia tem de ser tratada como falha.
2. detetor	A Microsoft a +4.82% com o cartão a dizer “ <i>an ordinary day</i> ”. O z era +1.11, abaixo do limiar, mas só 5 dos últimos 249 dias se tinham movido tanto. Duas réguas a discordar, e a interface escolhia em silêncio a mais tranquilizadora.
3. relevância	Sem o filtro, a mesma história contava duas vezes e um escritório de advogados era notícia da AMD.
5. precedentes	A deduplicação era de <i>texto exacto</i> : a mesma história escrita por dois meios continuava a contar como duas observações, e o alerta afirmava mais evidência do que tinha.

Os erros que cada fase custou (2/2)

fase	o erro, e porque é que passou despercebido
6. impacto	O impacto é medido por (empresa, dia), portanto três títulos do mesmo dia partilham o mesmo valor <i>por construção</i> . Em 36.8% dos alertas os casos assentavam em menos dias distintos do que casos mostrados: o alerta afirmava mais evidência independente do que tinha.
7. decomposição	O motor era escolhido pela maior componente em módulo , e uma empresa a subir com o setor a puxar ao contrário saía descrita como “foi o setor”. O sinal importa, e o módulo deita-o fora.
9. calibração	O ajuste é feito numa validação a 47.0% de prevalência e aplicado a um teste a 37.8%. A ordenação não muda, mas os limiares implantados assentam nessa escala, e por isso a probabilidade média prevista sai 0.050 acima da observada.
10. observação	Os dois registos que alimentam esta fase eram escritos em disco local, num contentor cujo disco é efêmero e pertence a outro processo. Estiveram parados dezanove dias sem um único erro registado , e foi encontrado a olhar, não por um alarme.

O padrão é sempre o mesmo: cada peça cumpria o seu contrato, e o que estava errado era uma suposição sobre a ligação entre elas que ninguém tinha escrito em lado nenhum.

É uma pergunta possível, e a resposta honesta é melhor do que a evasiva.

Existe em investigator/intelligence/ e investigator/narrator/ uma camada que usa um modelo de linguagem para escrever prosa, com uma verificação que rejeita qualquer frase cujos números não estejam ancorados num facto citado. Foi construída, foi atacada por um exercício adversário, e **23 de 23 ataques foram bloqueados**.

E está desligada em produção, de propósito.

Duas razões, e as duas estão no documento. A primeira é de posição: o trabalho defende que uma frase gerada é uma *afirmação*, e este sistema entrega afirmações com a evidência ao lado. A segunda é de honestidade: a dissertação **não reivindica** essa camada em lado nenhum, e manter no ar aquilo que o documento não descreve seria dívida.

A frase a dizer: *“está lá, funciona, e decidi não a expor porque contradiz a posição que o trabalho assume. As rotas foram retiradas e a configuração está a falso.”*

Se te esquecerem de tudo o resto, é isto que a construção ensina.

- 1 Cada fase tem **uma linha** onde a garantia é feita, e quase sempre é uma fatia, uma guarda ou a escolha de que dados entram. Não é o algoritmo.
- 2 Os erros que custaram mais tempo **não davam erro nenhum**. Davam um resultado plausível.
- 3 A parte difícil não foi escolher a técnica. Foi montar a avaliação que **consegue dizer que não**.

Guia irmão: tese/guia/ ensina a defender.
Este ensina a construir. Levas os dois.